

CSE 4351: Computer Security



Instructor: Dr. MD Armanuzzaman

Access Control II



Review

- **Access control** can be implemented in different ways
- **Discretionary access control**
 - lets subjects to grant privileges to other subjects at their discretion
- **Mandatory access control**
 - enforces system-wide policy
- **Role-based access control**
- **Attribute-based access control**

Confidentiality Policies

- Confidentiality policies emphasize the protection of confidentiality. The importance of these policies lies in part in what they provide, and in part in their role in the development of the concept of security.
- Bell-LaPadula Model
 - Military-style classifications. It has influenced the development of many other models and indeed much of the development of computer security technologies.

Bell-LaPadula Model

- The goal of the Bell-LaPadula security model is to prevent information flowing from **objects** at a security classification higher than a subject's clearance to that subject.

Let $L(S) = l_s$ be the security clearance of subject S , and let $L(O) = l_o$ be the security classification of object O . For all security classifications $l_i, i = 0, \dots, k-1$, $l_i < l_{i+1}$:

Simple Security Condition, Preliminary Version: S can read O if and only if $l_o \leq l_s$ and S has discretionary read access to O .

TOP SECRET (TS)	Tamara, Thomas	Personnel Files
SECRET (S)	Sally, Samuel	Electronic Mail Files
CONFIDENTIAL (C)	Claire, Clarence	Activity Log Files
UNCLASSIFIED (UC)	Ulaley, Ursula	Telephone List Files

Bell-LaPadula Model - Star Property

***-Property (Star Property), Preliminary Version:** S can write O if and only if $l_o \geq l_s$ and S has discretionary write access to O .

Because the activity log files are classified C and Tamara has a clearance of TS, she cannot write to the activity log files.

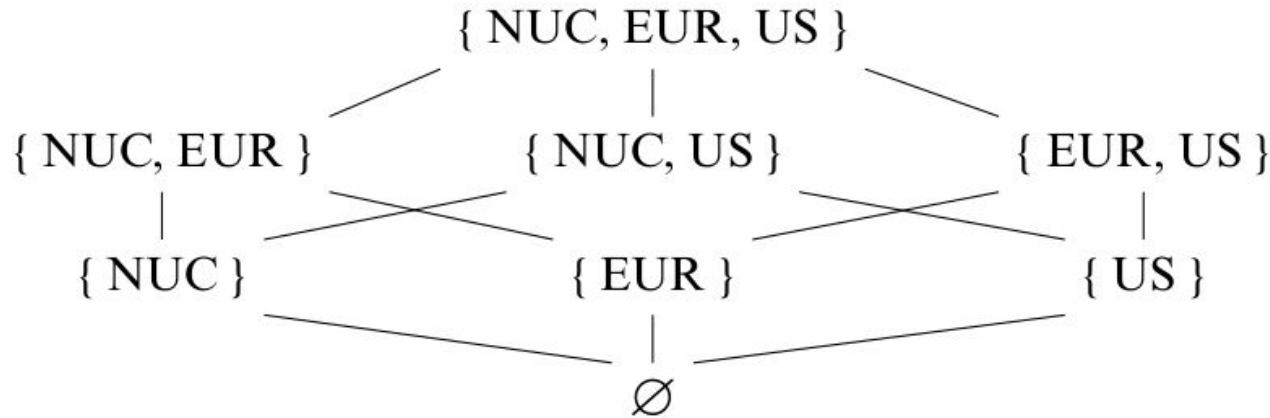
If both the simple security condition, preliminary version, and the *-property, preliminary version, hold, call the system a *secure system*. A straightforward induction establishes the following theorem.

Theorem 5.1. Basic Security Theorem, Preliminary Version: Let S be a system with a secure initial state s_0 , and let T be a set of state transformations. If every element of T preserves the simple security condition, preliminary version, and the *-property, preliminary version, then every state $s_i, i \geq 0$, is secure.

Bell-LaPadula Model - Extension

- Expand the model by adding a set of categories to each security classification. Each category describes a kind of information.
- Objects placed in multiple categories have the kinds of information in all of those categories. These categories arise from the “need to know” principle, which states that no subject should be able to read objects unless reading them is necessary for that subject to perform its functions.
- The sets of categories to which a person may have access is simply the power set of the set of categories.

Bell-LaPadula Model - Extension



Lattice generated by the categories NUC, EUR, and US. The lines represent the ordering relation induced by $\subseteq$

Lattice (order), a partially ordered set with unique least upper bounds and greatest lower bounds

Bell-LaPadula Model - Extension

Definition 5–1. The security level (L, C) dominates the security level (L', C') , written $(L, C) \text{ dom}(L', C')$, if and only if $L' \leq L$ and $C' \subseteq C$.

We write $(L, C) \neg\text{dom}(L', C')$ when $(L, C) \text{ dom}(L', C')$ is false. This relation also induces a lattice on the set of security levels [534].

EXAMPLE: George is cleared into security level $(\text{SECRET}, \{ \text{NUC}, \text{EUR} \})$, DocA is classified as $(\text{CONFIDENTIAL}, \{ \text{NUC} \})$, DocB is classified as $(\text{SECRET}, \{ \text{EUR}, \text{US} \})$, and DocC is classified as $(\text{SECRET}, \{ \text{EUR} \})$. Then:

- George *dom* DocA as $\text{CONFIDENTIAL} \leq \text{SECRET}$ and $\{ \text{NUC} \} \subseteq \{ \text{NUC}, \text{EUR} \}$
- George $\neg\text{dom}$ DocB as $\{ \text{EUR}, \text{US} \} \not\subseteq \{ \text{NUC}, \text{EUR} \}$
- George *dom* DocC as $\text{SECRET} \leq \text{SECRET}$ and $\{ \text{EUR} \} \subseteq \{ \text{NUC}, \text{EUR} \}$

Bell-LaPadula Model - Extension

Simple Security Condition: S can read O if and only if $S \text{ dom } O$ and S has discretionary read access to O .

In the previous example, George can read DocA and DocC but not DocB (again, assuming that the discretionary access controls allow such access).

Suppose Paul is cleared into security level (SECRET, { EUR, US, NUC }) and has discretionary read access to DocB. Paul can read DocB; were he to copy its contents to DocA and set its access permissions accordingly, George could then read DocB. The modified *-property prevents this:

***-Property:** S can write to O if and only if $O \text{ dom } S$ and S has discretionary write access to O .

DocA dom Paul is false (because $C(\text{Paul}) \not\subseteq C(\text{DocA})$), so Paul cannot write to DocA.

The simple security condition is often described as “no reads up” and the *-property as “no writes down.”

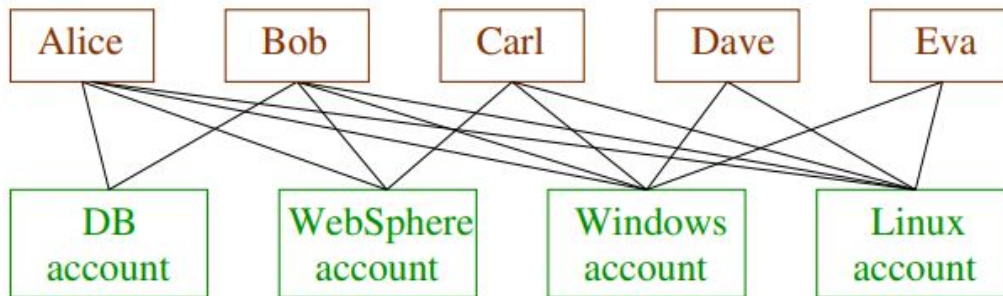
Redefine a *secure system* to be a system in which both the simple security property and the *-property hold. The analogue to the Basic Security Theorem, preliminary version, can also be established by induction.

Role-Based Access Control

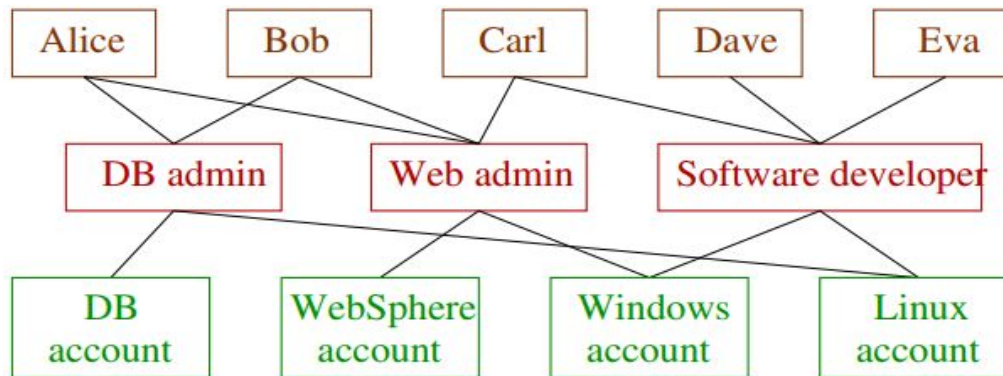
- In **Role-Based Access Control (RBAC)** models, subjects are combined into “roles” according to their privileges in the organization
 - often based on job function
- Permissions are assigned to roles rather than users
- A user can assume one or more roles within the organization according to their responsibilities
- RBAC fits operational model of an organization and is widely used

Role-Based Access Control

- Non-role-based AC



- Role-based AC



Role-Based Access Control

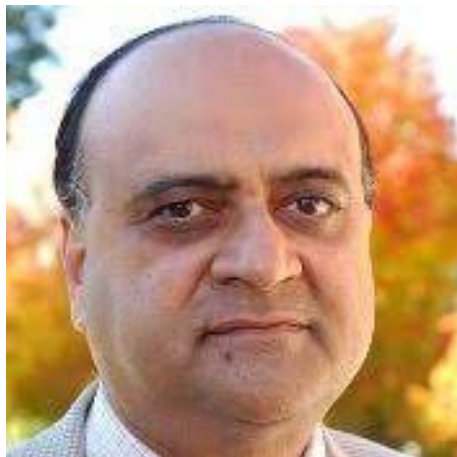
- Motivation for RBAC
 - problem: it is difficult to administer user-permission relation
 - roles are a level of indirection
 - “All problems in Computer Science can be solved by another level of indirection” B. Lampson
- RBAC is
 - multi-faceted
 - Multi-dimensional
 - open ended
 - ranging from simple to sophisticated

Role-Based Access Control

- Why use roles?
 - fewer relationships to manage
 - potential decrease from $O(mn)$ to $O(m + n)$, where m is the number of users and n is the number of permissions
 - there are often more users than roles and more objects than roles
 - roles are a useful level of abstraction
 - organizations operate based on roles
 - roles are likely to be more stable than the set of users and the set of resources
 - roles can effectively implement the principle of least privilege
 - finding the minimum set of necessary access rights is performed per role rather than per subject

Groups vs. Roles

- How are roles different from groups?
 - A group is a collection of users, rather than a collection of permissions.
 - Another aspect of RBAC that distinguishes it from traditional group mechanisms is the concept of a **session**, which allows activation of a subset of roles assigned to a user.



Ravi Sandhu
UTSA

Feature

Role-Based Access Control Models

Ravi S. Sandhu
*George Mason University and
SETA Corporation*

Edward J. Coyne
Hal L. Feinstein
Charles E. Youman
SETA Corporation

Starting in the 1970s, computer systems featured multiple applications and served multiple users, leading to heightened awareness of data security issues. System administrators and software developers alike focused on different kinds of access control to ensure that only authorized users were given access to certain data or resources. One kind of access control that emerged is role-based access control (RBAC).

A role is chiefly a semantic construct forming the basis of access control policy. With RBAC, system administrators create roles according to the job functions performed in a company or organization, grant permissions (access authorization) to those roles, and then assign users to the roles on the basis of their specific job responsibilities and qualifications (see sidebar "Role-based access control terms and concepts").

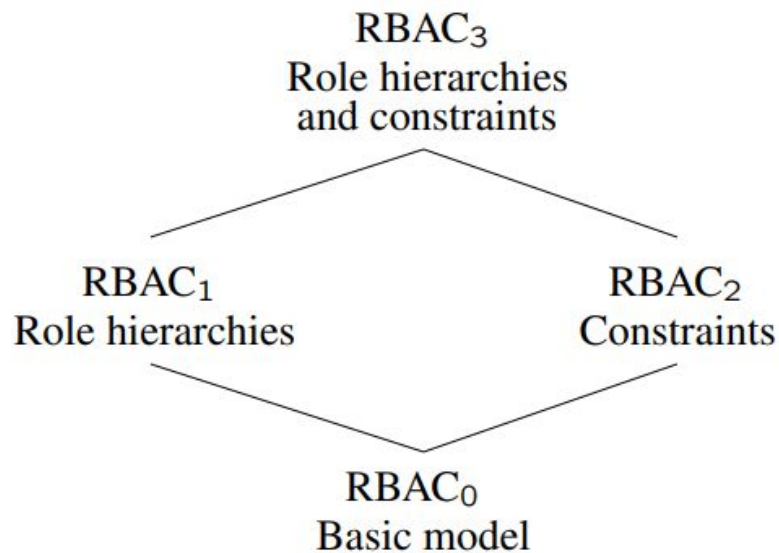
A role can represent specific task competency, such as that of a physician or a pharmacist. A role can embody the authority and responsibility of, say, a project supervisor. Authority and responsibility are distinct from competency. A person may be competent to manage several departments but have the responsibility for only the department actually managed. Roles can also reflect specific duty assignments rotated through multiple users—for example, a duty physician or a shift manager. RBAC models and implementations should conveniently accommodate all these manifestations of the role concept.

Roles define both the specific individuals allowed to access resources and the extent to which resources are accessed. For example, an operator role might access all computer resources but not change access permissions; a security-officer role might change permissions but have no access to resources; and an auditor role might access only audit trails. Roles are used for system administration in such network operating systems as Novell's NetWare and Microsoft's Windows NT.

The particular combination of users and permissions brought together by a role tend to change over time. The permissions associated with a role, on the other hand, are more stable; they tend to change less often than the people who fill the job function that role represents. Therefore, basing security administration on roles rather than on permissions is simpler.

RBAC Models

- The family of RBAC models proposed by Sandhu et al. (1996)

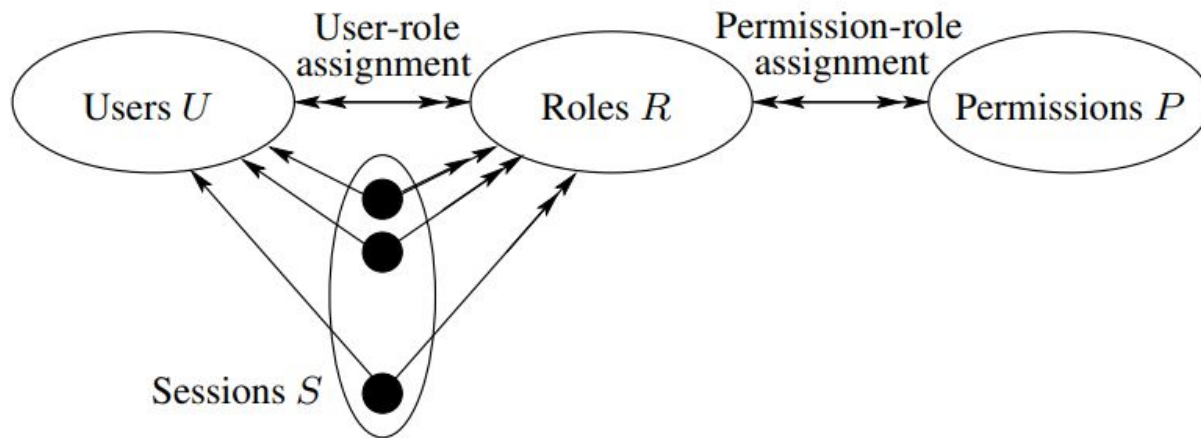


RBAC₀: the formal core

- RBAC₀ contains four types of entities
 - users U
 - roles R
 - permissions P
 - sessions S
- User assignment is many-to-many $UA \subseteq U \times R$
- Permission assignment is many-to-many $PA \subseteq P \times R$
- Session activation
 - one-to-one for user: $S \rightarrow U$
 - one-to-many for roles: $S \rightarrow 2^R$

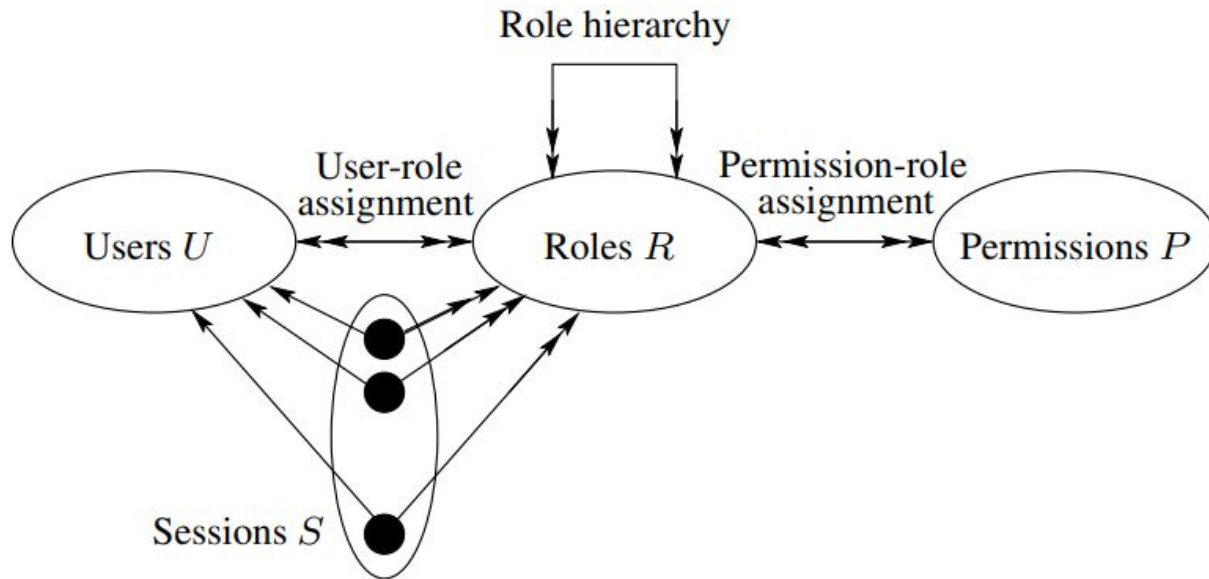
RBAC₀: the formal core

- A session s must comply with UA and PA assignments
 - $roles(s) \subseteq \{r \mid (user(s), r) \in UA\}$
 - permissions of session s are $\bigcup_{r \in roles(s)} \{p \mid (p, r) \in PA\}$



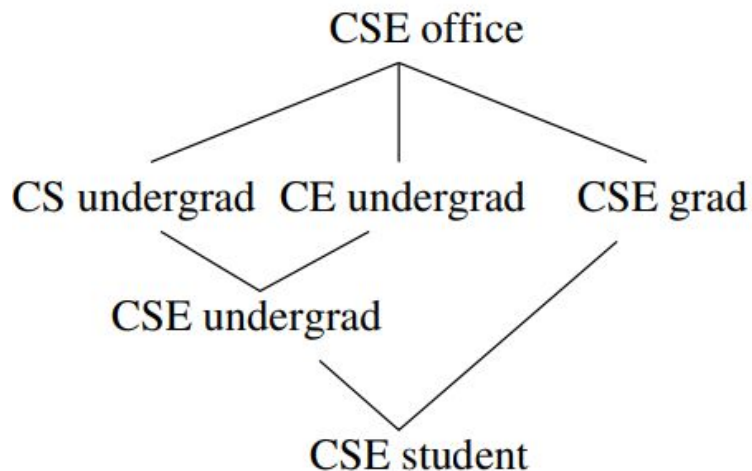
RBAC₁: role hierarchies

- RBAC₁ enhances RBAC₀ with role hierarchies



RBAC₁: role hierarchies

- **Role hierarchies** are based on the idea that subordinate job functions may have a subset of access rights of a superior job function
 - a role inherits access rights of its descendant roles
- **Example** of a role hierarchy



RBAC₁: role hierarchies

- Formal model:
 - U, P, R, S, PA, UA are unchanged from RBAC₀
 - role hierarchy $RH \subseteq R \times R$ is a partial order on R written as $\geq$
 - $r1 \geq r2$ means that $r1$ is an ancestor of $r2$
 - partial order means that relationship between any two roles can be undefined
 - requirements on session activation change
 - $roles(s) \subseteq \{r \mid \exists r' \text{ s.t. } (r' \geq r) \ \& \ (user(s), r') \in UA\}$
 - session s has permissions

$$U_{r \in roles(s)} \{p \mid \exists r' \text{ s.t. } (r \geq r') \ \& \ (p, r') \in PA \}$$

RBAC₂

- No formal model is specified for RBAC₂ that adds constraints to RBAC0
- A **constraint** is a condition related to roles or a relationship defined on roles
- **Types of constraints** (Sandhu et al. 96)
 - mutually exclusive roles
 - cardinality constraints
 - prerequisite constraints

Constraints in RBAC

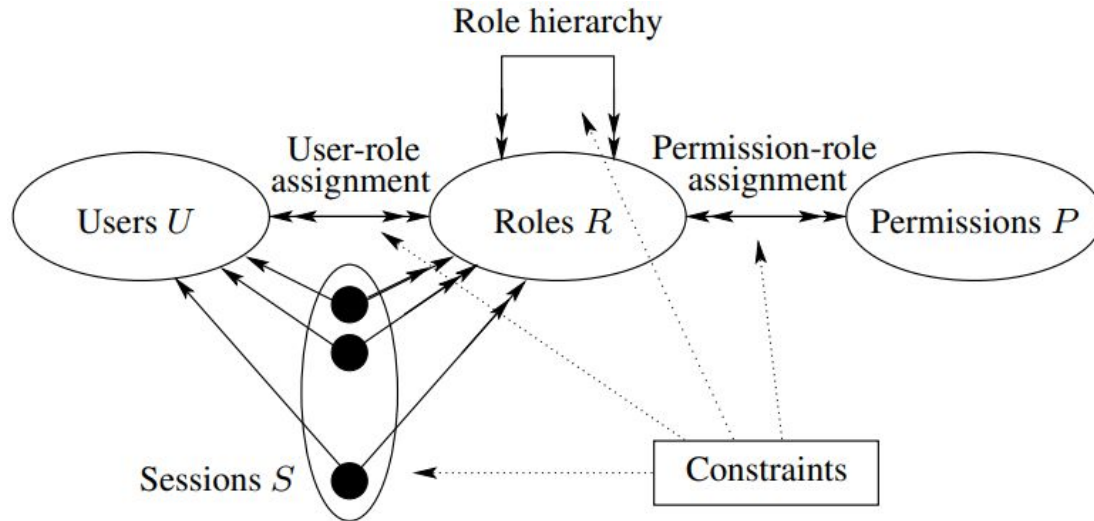
- **Mutually exclusive roles**: a user can be assigned to only one role from a particular set of roles
 - static exclusion:
 - dynamic exclusion:
 - such constraints support the separation of duties principle
- **Prerequisite** (or precondition) constraints: the prerequisite must be true before a user can be assigned to a particular role
 - a user can be assigned to role r_1 only if it is already assigned to another role r_2

Constraints in RBAC

- **Cardinality constraints:** setting restrictions on the number of roles
 - user-role assignment
 - at most k users can be assigned to the role
 - a user can be assigned to at most m roles
 - role-permission assignment
 - role activation

RBAC₃: constraints, and the separation-of-duty callback

- RBAC₃: features of RBAC₀, RBAC₁, and RBAC₂



- Now role constraints can be based on the role hierarchy

RBAC in Use

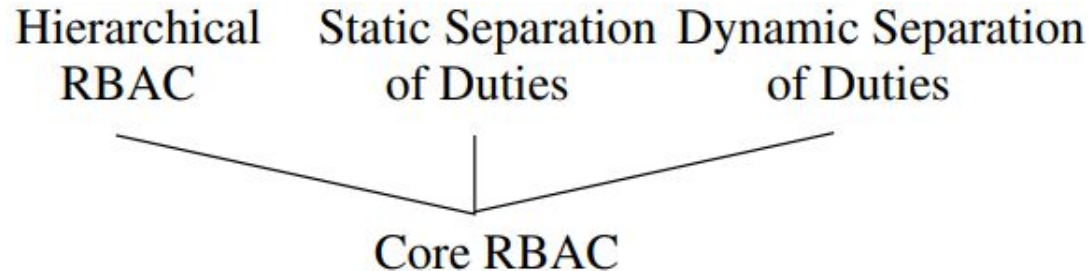
- Products that use RBAC
 - database management systems (e.g., Oracle)
 - enterprise security management (e.g., IBM Tivoli Identity Manager)
 - operating systems (e.g., Solaris OS, AIX)
- RBAC economic impact study (2002)
 - was conducted by the Research Triangle Institute (RTI) based on interviews with software developers and companies that use RBAC
 - it estimated by 2006 30–50% of employees in service sector would be managed by RBAC systems (10–25% for non-service sectors)
 - it conservatively estimated the economic benefits of this degree of penetration through 2006 to be \$671 million

RBAC in Use

- Another analysis was performed in 2010
 - RBAC use rose to 41% in 2009 and was estimated to be just over 50% in 2010
 - over 80% of respondents reported that using roles improved efficiency of maintaining their organization's access control policy
 - economic benefits of RBAC adoption between 1994 and 2009 were estimated at \$6 billion

The RBAC Standard

- In 2001 RBAC was proposed to become a NIST standard
- It was adopted as ANSI (American National Standards Institute) standard 359 in 2004
- The standard has the following structure



The RBAC Standard

- The ANSI standard has been criticized by Li et al. (2007)
 - there are many errors
 - there are other limitations and design flaws
 - the publication proposes several changes to the standard
- It was republished as 359-2012 and since reaffirmed as 359-2017 (R2017)
 - the current version consists of two parts: the RBAC reference model and the RBAC system and administrative functional specification

RBAC Extensions

- RBAC has been extensively studied
 - many extensions exist (temporal, geo-spatial, privacy-aware)
 - administration of RBAC
 - constraints, workflow, role engineering

Attribute-Based Access Control

- **Attribute-based access control** (ABAC) is a rather recent mechanism for specifying and enforcing access control
 - properties are specified in the form of attributes
 - authorizations involve evaluating predicates on attributes
 - conditions on properties of both the subject and resource can be enforced

Attribute-Based Access Control

- ABAC provides a lot of **flexibility** in specifying rules and supports fine-grained access control
 - it is capable of enforcing DAC, MAC, and RBAC concepts
- This comes at a **performance cost**
 - it has seen the most success for web services and cloud computing where there is already a response delay
- There are **three key elements** in an ABAC model
 - attributes
 - policies
 - architecture

Attribute-Based Access Control

- ABAC **attributes** are characteristics of subjects, objects, environment, and operations preassigned by an authority
- An ABAC model can have three types of attributes
 - subject attributes
 - e.g., name, ID, job function, etc.
 - object attributes
 - e.g., name/title, creation time, ownership information, etc.
 - environment attributes
 - e.g., current date and time, network's security level, etc

Attribute-Based Access Control

- ABAC **architecture** specifies how access control is enforced
- When a user submits an access request, the authorization decision is governed by
 - access control policies
 - subject attributes
 - object attributes
 - environmental attributes
- Contrast the above with ACLs in DAC
- ABAC systems are thus significantly more complex

Attribute-Based Access Control

- ABAC **policies** rules implement authorizations using subject-object-environment information (s, o, e)
 - there may not be explicit roles or groups and authorization decisions are instead made based on attributes
 - e.g., consider access to a database of movies
 - everyone can access movies rated as G
 - users of age ≥ 13 can access movies rated as PG-13
 - users of age ≥ 17 can access movies rated as R
 - a policy might be written as $P1(s, o, e)$:

$$\text{return } (\text{Age}(s) \geq 17 \wedge \text{Rating}(o) \in \{R, PG-13, G\}) \vee (13 \leq \text{Age}(s) < 17 \wedge \text{Rating}(o) \in \{PG-13, G\}) \vee (\text{Age}(s) < 13 \wedge \text{Rating}(o) \in \{G\})$$

Attribute-Based Access Control

- ABAC **policies** can be combined into more complex rules
 - e.g., limit access to new releases to premium membership
 - $P2(s, o, e): \text{return } (\text{MemberType}(s) = \text{Premium})$
 $\vee (\text{MemberType}(s) = \text{Regular} \wedge \text{MovieType}(o) = \text{OldRelease})$
 - grant access if both rules are met
 - $P3(s, o, e): \text{return } P1(s, o, e) \wedge P2(s, o, e)$
 - the environment (e.g., the date) can be used for policies such as promotions

Identity Management

- **Identity management** is related, but not identical to access control
 - it refers to maintaining identity independent of one's job title, job duties, access privileges, location, etc.
 - contrast this with accounts to login into applications, networks, etc.
- A digital identity is typically established based on a set of **attributes**
 - the attributes together comprise a unique user within a system or enterprise
 - **credentials** get associated with an identity
 - **access** is based on credentials that an identity possesses

Identity Management

- Can you use identities maintained by one organization to access systems maintained by other organizations?
 - **identity federation** refers to the technology, policies and processes to enable this functionality
 - it answers this question via trust
- When disclosing an identity's attributes and credentials to external parties, we generally want to follow the **need-to-know principle**
- Traditionally identities were maintained by **identity service providers** which **relying parties can use**
- More recently, **trust network providers** regulate interactions between identity service providers and relying parties

Identity Management

- **OpenID** is an open standard that allows users to be authenticated by relying parties using third party OpenID identity providers
- **Open Identity Trust Framework** (OITF) is a standardized specification of a trust framework for identity and attribute exchange
 - it was developed by the community and nonprofit organizations
- **Attribute Exchange Network** (AXN) is an online gateway for identity service providers and relying parties to access verified identity attributes

Summary

- The choice of an access control model depends on the context
 - system requirements, security policies, etc.
 - can use DAC, MAC, RBAC, attribute-based AC, or other solutions
 - have to consider costs of implementation, maintenance, and rule enforcement
- Federated identity allows for identity credentials to be used across different organizations